

Klassiker 13a: ggT (Euklid)

Zum Foliensatz 13a — Algorithmen modellieren · SoSe 2026

Dorian Zwanzig

Die Aufgabe

Veggie Soles packt 48 Kartons Eco-Sneaker und 18 Kartons Hemp-High zu **gleich großen, sortenreinen Bündeln**. Die größte mögliche Bündelgröße ist der **größte gemeinsame Teiler (ggT)**. Schreibe eine Funktion `ggt(a, b)`, die den ggT nach **Euklids Algorithmus** berechnet — übersetze dazu das Flussdiagramm vom Foliensatz (Raute mit Rücksprung, Rumpf, Ausgabe) in Python. Gib das Ergebnis für die beiden Beispielpaare aus.

Erwartetes Verhalten

```
ggT von 48 und 18: 6
ggT von 135 und 50: 5
```

Erlaubte Bausteine

Alles bis Notebook 13a — mehr brauchst du nicht: Variablen und f-Strings · `print()` · Rest-Operator `%` · Vergleich `!=` · `while` · `def` / `return`

Gestufte Hinweise

Erst selbst probieren — dann Hinweis für Hinweis aufdecken.

Hinweis 1 (Ansatz): Euklids Idee: Der ggT von a und b ist derselbe wie der ggT von b und dem Rest $a \% b$. Wiederhole das, bis b null ist — dann steht der ggT in a . Die Raute im Flussdiagramm ist deine `while`-Bedingung.

Hinweis 2 (Struktur): Pseudocode:

```
ALGORITHMUS GGT(a, b)
  SOLANGE b != 0:
    rest <- a MOD b
    a <- b
    b <- rest
  RUECKGABE a
```

Hinweis 3 (der Kniff): Die **Reihenfolge im Rumpf** ist entscheidend: Erst den Rest in `rest` **zwischenspeichern**, dann `a = b`, dann `b = rest`. Wer `b` zuerst überschreibt, verliert den Wert, den `a` noch braucht — dasselbe Problem wie beim Variablentausch (Klassiker 04, Hilfsvariable!).

Bonus

Eingabe-Variante: Lies a und b mit `int(input(...))` ein. **Kontrolle:** `import math`, dann `math.gcd(48, 18)` — liefert deine Funktion dasselbe? **Trace:** Schreibe für `ggt(48, 18)` eine Tabelle mit a und b pro Durchlauf ($48/18 \rightarrow 18/12 \rightarrow 12/6 \rightarrow 6/0$) — warum *muss* die Schleife enden?

Musterlösung nach der Vorlesung: Praesentationen_src/13a_Algorithmen_Modellierung/90_klassiker_ggt.py
· Vertiefung: Notebook 13a, Beispiel 5 und Kapitel 4.